

OSSP-PRACTICAL-WEEK7

TASK1: Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

void show_addresses()
{
    int stack_var = 20;
    static int static_var = 30;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 40;

    printf("Process Address Space\n");
    printf("-----\n");

    printf("Code   address : %p\n", (void *)show_addresses);
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap   address : %p\n", (void *)heap_var);
    printf("Stack  address : %p\n", (void *)&stack_var);

    printf("\nProcess ID: %d\n", getpid());

    printf("\nPress Enter to exit...");
    getchar();

    free(heap_var);
}

int main()
{
    show_addresses();

    return 0;
}
```

Output:

```
root@LAPTOP-1C0JKKG6:~# nano practical_week7_task1.c
root@LAPTOP-1C0JKKG6:~# gcc practical_week7_task1.c -o practical_week7_task1
root@LAPTOP-1C0JKKG6:~# ./practical_week7_task1
Process Address Space
-----
Code   address : 0x5f13ff919209
Global address : 0x5f13ff91c010
Static address : 0x5f13ff91c014
Heap   address : 0x5f143bf452a0
Stack  address : 0x7ffe4ae2d78c

Process ID: 540

Press Enter to exit...|
```

Observation: I learned how a Linux process is organized into different memory segments such as code, global, static, heap, and stack. I learned how to print the memory addresses of variables belonging to these segments and understood that dynamically allocated memory is stored in the heap while local variables are stored in the stack.

Task2: Make use of the /proc/[pid]/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Output:

```
root@LAPTOP-1COJKKG6:~# cat /proc/540/maps
5f13ff918000-5f13ff919000 r--p 00000000 08:30 36543 /root/practical_week7_task1
5f13ff919000-5f13ff91a000 r-xp 00001000 08:30 36543 /root/practical_week7_task1
5f13ff91a000-5f13ff91b000 r--p 00002000 08:30 36543 /root/practical_week7_task1
5f13ff91b000-5f13ff91c000 r--p 00002000 08:30 36543 /root/practical_week7_task1
5f13ff91c000-5f13ff91d000 rw-p 00003000 08:30 36543 /root/practical_week7_task1
5f143bf45000-5f143bf66000 rw-p 00000000 00:00 0 [heap]
7532cfe00000-7532cfe28000 r--p 00000000 08:30 49749 /usr/lib/x86_64-linux-gnu/libc.so.6
7532cfe28000-7532cffb0000 r-xp 00028000 08:30 49749 /usr/lib/x86_64-linux-gnu/libc.so.6
7532cffb0000-7532cffff000 r--p 001b0000 08:30 49749 /usr/lib/x86_64-linux-gnu/libc.so.6
7532cffff000-7532d0003000 r--p 001fe000 08:30 49749 /usr/lib/x86_64-linux-gnu/libc.so.6
7532d0003000-7532d0005000 rw-p 00202000 08:30 49749 /usr/lib/x86_64-linux-gnu/libc.so.6
7532d0005000-7532d0012000 rw-p 00000000 00:00 0
7532d018a000-7532d018d000 rw-p 00000000 00:00 0
7532d0192000-7532d0194000 rw-p 00000000 00:00 0
7532d0194000-7532d0198000 r--p 00000000 00:00 0 [vvar]
7532d0198000-7532d019a000 r--p 00000000 00:00 0 [vvar_vclock]
7532d019a000-7532d019c000 r-xp 00000000 00:00 0 [vdso]
7532d019c000-7532d019d000 r--p 00000000 08:30 49746 /usr/lib/x86_64-linux-gnu/ld-linux-x86_64.so.2
7532d019d000-7532d01c8000 r-xp 00001000 08:30 49746 /usr/lib/x86_64-linux-gnu/ld-linux-x86_64.so.2
7532d01c8000-7532d01d2000 r--p 0002c000 08:30 49746 /usr/lib/x86_64-linux-gnu/ld-linux-x86_64.so.2
7532d01d2000-7532d01d4000 r--p 00036000 08:30 49746 /usr/lib/x86_64-linux-gnu/ld-linux-x86_64.so.2
7532d01d4000-7532d01d6000 rw-p 00038000 08:30 49746 /usr/lib/x86_64-linux-gnu/ld-linux-x86_64.so.2
7ffe4ae0f000-7ffe4ae30000 rw-p 00000000 00:00 0 [stack]
root@LAPTOP-1COJKKG6:~# |
```

```
root@LAPTOP-1COJKKG6:~# cat /proc/540/status | grep -E "VmSize|VmRSS|VmData"
VmSize:      2688 kB
VmRSS:       1540 kB
VmData:       224 kB
VmStk:       132 kB
root@LAPTOP-1COJKKG6:~# |
```

Observation: I learned how to examine the virtual memory layout of a running Linux process using /proc/[PID]/maps. I understood how memory mappings represent code, data, heap, stack, and shared libraries, along with their read, write, and execute permissions. I also learned how tools such as pmap can be used to analyze process memory usage.